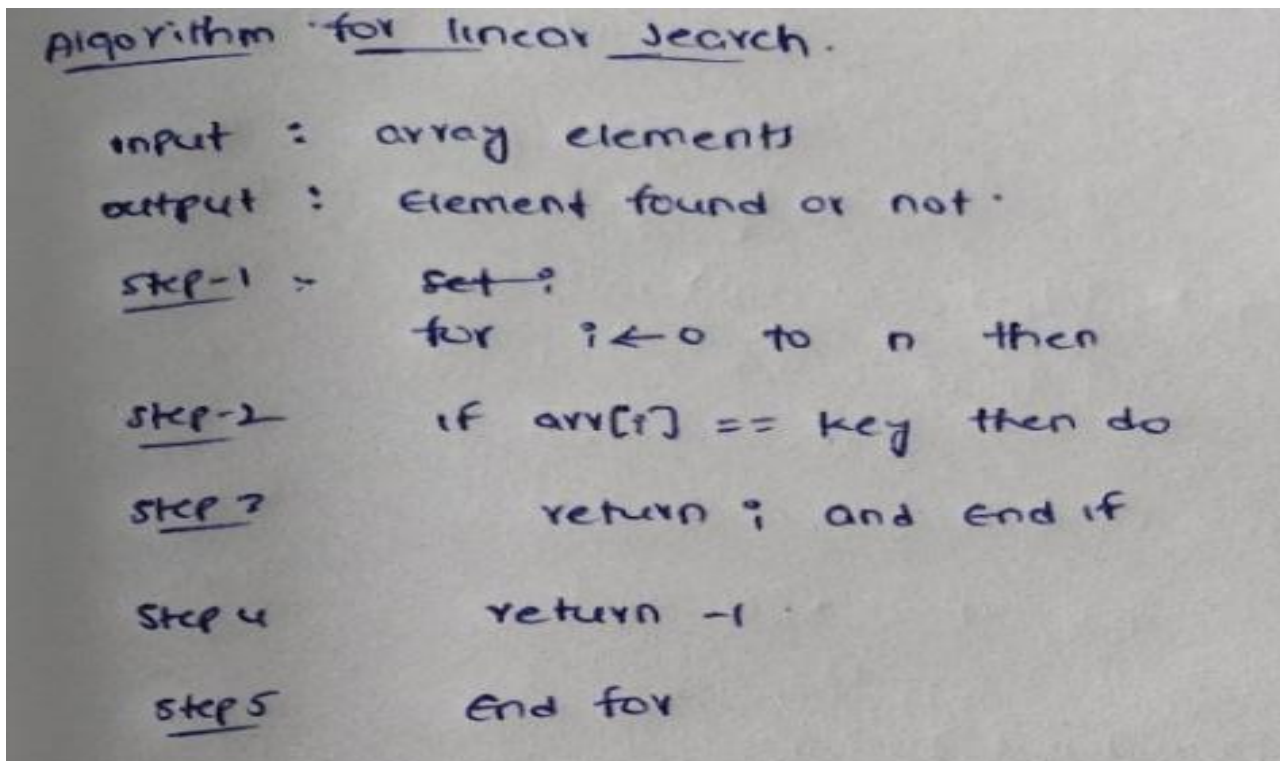


PRACTICAL-2

AIM: Implementation and Time analysis of linear and binary search algorithm.

LINEAR SEARCH

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;
int main()
{
    int n, arr[100], key;
    cout << "Enter size of array: ";
    cin >> n;
    cout << "Enter array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];
    cout << "Enter element to search: ";
    cin >> key;
    for(int i = 0; i < n; i++)
```

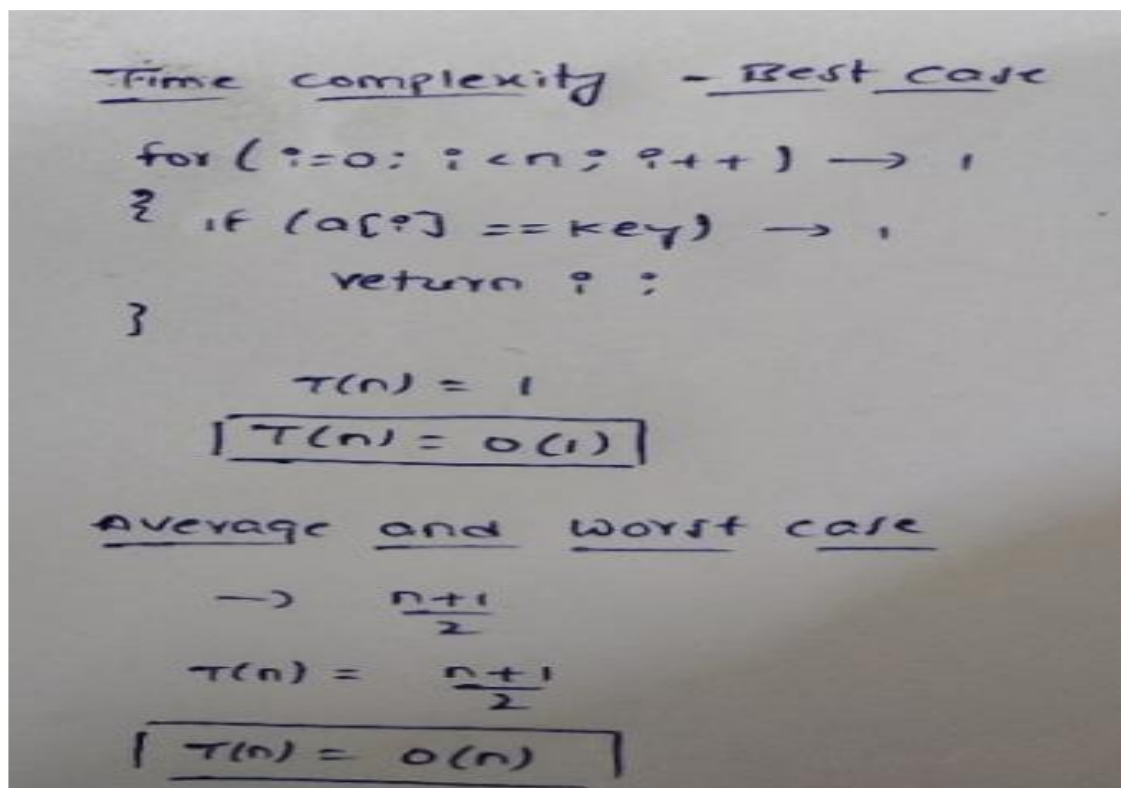
```
{  
if(arr[i] == key)  
{  
cout << "Element found at position " << i + 1;  
return 0;  
}  
}  
cout << "Element not found";  
return 0;  
}
```

OUTPUT:

Output

```
Enter size of array: 5  
Enter array elements: 3 45 6 7 8  
Enter element to search: 6  
Element found at position 3  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



BINARY SEARCH

ALGORITHM:

Algorithm for Binary search

input : sorted array

output : Element found or not.

step 1 : set $low = 0$, $high = n - 1$

step 2 : Repeat while $low \leq high$:

step 3 : $mid = (low + high) / 2$

step 4 : if $arr[mid] \geq key$ then do
return mid.

step 5 : else if $arr[mid] < key$ then do
 $low = mid + 1$;

step 6 : else $high = mid - 1$..

step 7 : End while.

step 8 : return -1

CODE:

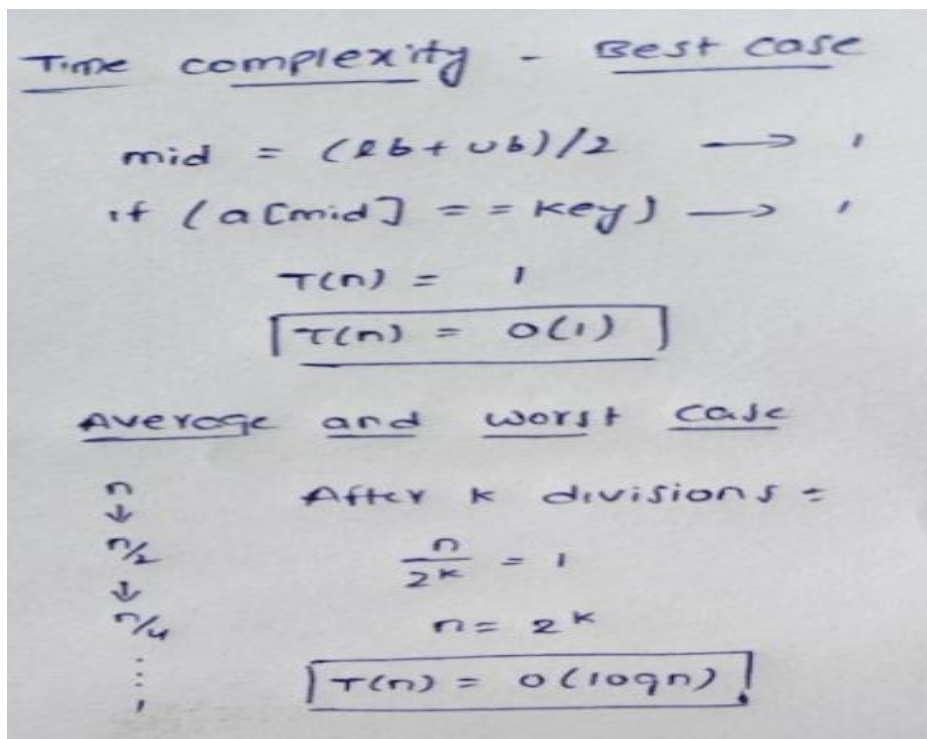
```
#include
<iostream> using
namespace std; int
main()
{ int n, arr[100], key; int low,
high, mid; cout << "Enter size
of array: "; cin >> n;
cout << "Enter sorted array elements: ";
for(int i = 0; i < n; i++) cin >>
arr[i]; cout << "Enter element to
search: "; cin >> key; low = 0;
high = n - 1;
while(low <= high)
{ mid = (low + high) /
2; if(arr[mid] == key)
{
cout << "Element found at position " << mid +
1;
return 0;
}
else if(arr[mid] < key)
{ low = mid +
1; } else {
high = mid -
1; }}
cout << "Element not found";
return 0;
}
```

OUTPUT:

```
Output
Enter size of array: 6
Enter sorted array elements: 3 4 5 6 7 66
Enter element to search: 66
Element found at position 6

=== Code Execution Successful ===
```

TIME COMPLEXITY:



CONCLUSION:

The practical successfully implemented and analyzed Linear Search and Binary Search algorithms. Linear Search checks each element sequentially and can be used on both sorted and unsorted data, whereas Binary Search repeatedly divides a sorted array into two halves to locate the required element.

From the time analysis, Linear Search has a best-case complexity of $O(1)$ and a worst-case complexity of $O(n)$, while Binary Search has a best-case complexity of $O(1)$ and a worst-case complexity of $O(\log n)$. Therefore, Binary Search is more efficient for large sorted datasets. This practical helped in understanding the working, implementation, and performance differences between the two searching techniques.